Assignment No. 8

Title: Implementation of Auto-Regressive Model

Objective:

To implement the Auto-Regressive (AR) Model using Python for time series forecasting and analyze how past observations influence the prediction of future values.

Problem Statement:

Implement the Auto-Regressive Model on a given time series dataset using Python. Load the dataset into a pandas DataFrame and apply the Auto-Regressive technique to analyze the relationship between current values and their past values. Use the model to forecast future values and visualize the results.

Theory:

Time Series Analysis

Time series analysis is a statistical technique used to analyze data points collected or recorded at specific time intervals. It helps in understanding patterns such as trends, seasonality, and cyclic variations in the data and is widely used for forecasting future values.

Examples of time series data include stock prices, weather data, sales records, and economic indicators.

Auto-Regressive (AR) Model

The Auto-Regressive (AR) model is a time series forecasting technique in which the current value of a variable is expressed as a linear combination of its previous values and a random error term.

The term **Auto-Regressive** indicates that the variable is regressed on its own past values.

Mathematical Representation

The AR(p) model can be expressed as:

$$Y_t = c + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \ldots + \phi_p Y_{t-p} + \epsilon_t$$

Where:

- Y_t = Value at time **t**
- c = Constant term
- $Y_{t-1}, Y_{t-2} \ldots Y_{t-p}$ = Past observations (lags)
- p = Order of the model (number of lag values)
- ϵ_t = Random error term

The AR model assumes that past values have a significant influence on the present value.

Applications of Auto-Regressive Model

1. Stock market price prediction
2. Weather forecasting
3. Economic data analysis
4. Demand forecasting
5. Signal processing

Implementation using Python

Step 1: Import Required Libraries

Python libraries such as **Pandas**, **Matplotlib**, and **Statsmodels** are used for time series analysis and visualization.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.ar_model import AutoReg
from sklearn.metrics import mean_squared_error
```

- **Pandas** is used for data manipulation and analysis.
- **NumPy** is used for numerical computations.
- **Matplotlib** is used for plotting graphs.
- **Statsmodels** provides statistical models for time series analysis.

Step 2: Load the Dataset

The dataset is loaded using the pandas **read_csv()** function.

```
df = pd.read_csv("data.csv")
```

This loads the dataset into a pandas DataFrame.

Step 3: Convert Date Column into Datetime Format

```
df['Date'] = pd.to_datetime(df['Date'])
```

This converts the Date column into datetime format so that it can be used in time series analysis.

Step 4: Set Date Column as Index

```
df.set_index('Date', inplace=True)
```

The Date column is set as the index to organize the dataset based on time.

Step 5: Select the Target Variable

```
data = df['Close']
```

Here the **closing price** is selected as the time series variable for forecasting.

Step 6: Split the Dataset into Training and Testing Data

```
train_size = int(len(data) * 0.8)
train, test = data[:train_size], data[train_size:]
```

The dataset is divided into:

- **Training data (80%)** for building the model
- **Testing data (20%)** for evaluating the model

Step 7: Create the Auto-Regressive Model

```
model = AutoReg(train, lags=1)
```

This creates an Auto-Regressive model with **lag = 1**, meaning the current value depends on the previous value.

Step 8: Fit the Model

```
model_fit = model.fit()
```

This trains the model using the training dataset.

Step 9: Generate Predictions

```
predictions = model_fit.predict(start=len(train), end=len(train)+len(test)-1)
```

This generates predictions for the test dataset.

Step 10: Plot the Results

```
plt.figure(figsize=(12,6))
plt.plot(train.index, train, label="Train Data")
plt.plot(test.index, test, label="Actual Data")
plt.plot(test.index, predictions, label="Predicted Data")
plt.legend()
plt.title("Auto-Regressive Model Forecast")
plt.xlabel("Date")
plt.ylabel("Value")
plt.show()
```

This graph compares the **actual values** with the **predicted values**.

Step 11: Evaluate the Model

```
rmse = np.sqrt(mean_squared_error(test, predictions))
print("RMSE:", rmse)
```

RMSE (Root Mean Square Error) is used to measure the difference between actual values and predicted values.

Lower RMSE indicates better prediction accuracy.

Advantages of Auto-Regressive Model

1. Easy to implement and understand.
2. Captures relationships between past and current values.
3. Useful for short-term forecasting.
4. Widely used in financial and economic analysis.

Limitations of Auto-Regressive Model

1. Assumes the time series is stationary.
2. Cannot capture complex nonlinear relationships.
3. Performance decreases with highly volatile data.

Conclusion

Thus, the Auto-Regressive Model was successfully implemented using Python. The model used past observations to predict future values in the time series dataset. The results were visualized using graphs and evaluated using RMSE. The Auto-Regressive model is a simple yet powerful technique used for time series forecasting in many real-world applications.